Progettazione di Software Sicuro

Elvinia Riccobene

Prodotto e Processo

Prodotto e processo

Prodotto: *cosa* realizziamo (software)

Processo: *come* realizziamo il prodotto

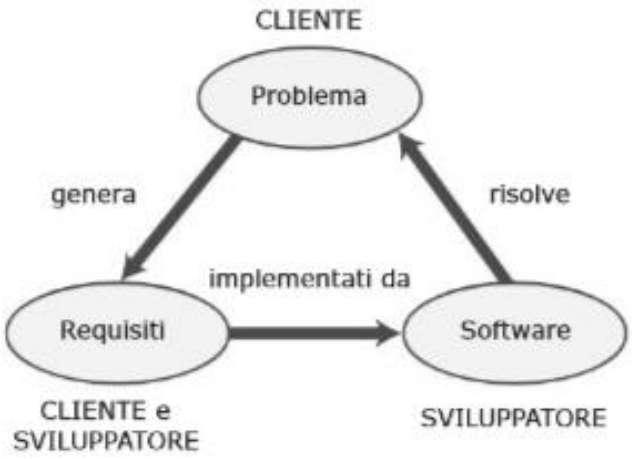
- Esistono diversi modelli in cui organizzare le fasi di sviluppo

Entrambi sono estremamente importanti

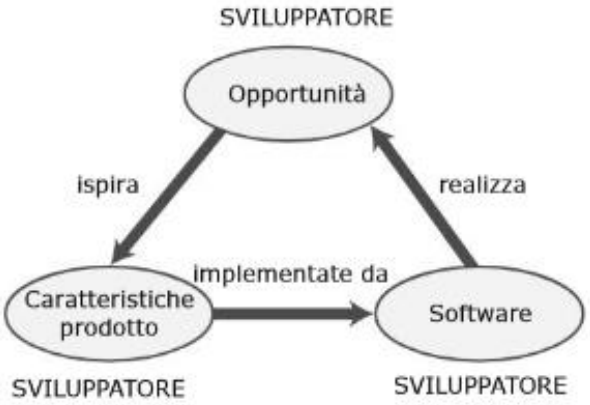
Entrambi hanno delle qualità

Diverse *origini* del *Prodotto* e diversi *processi* di Ing del SW

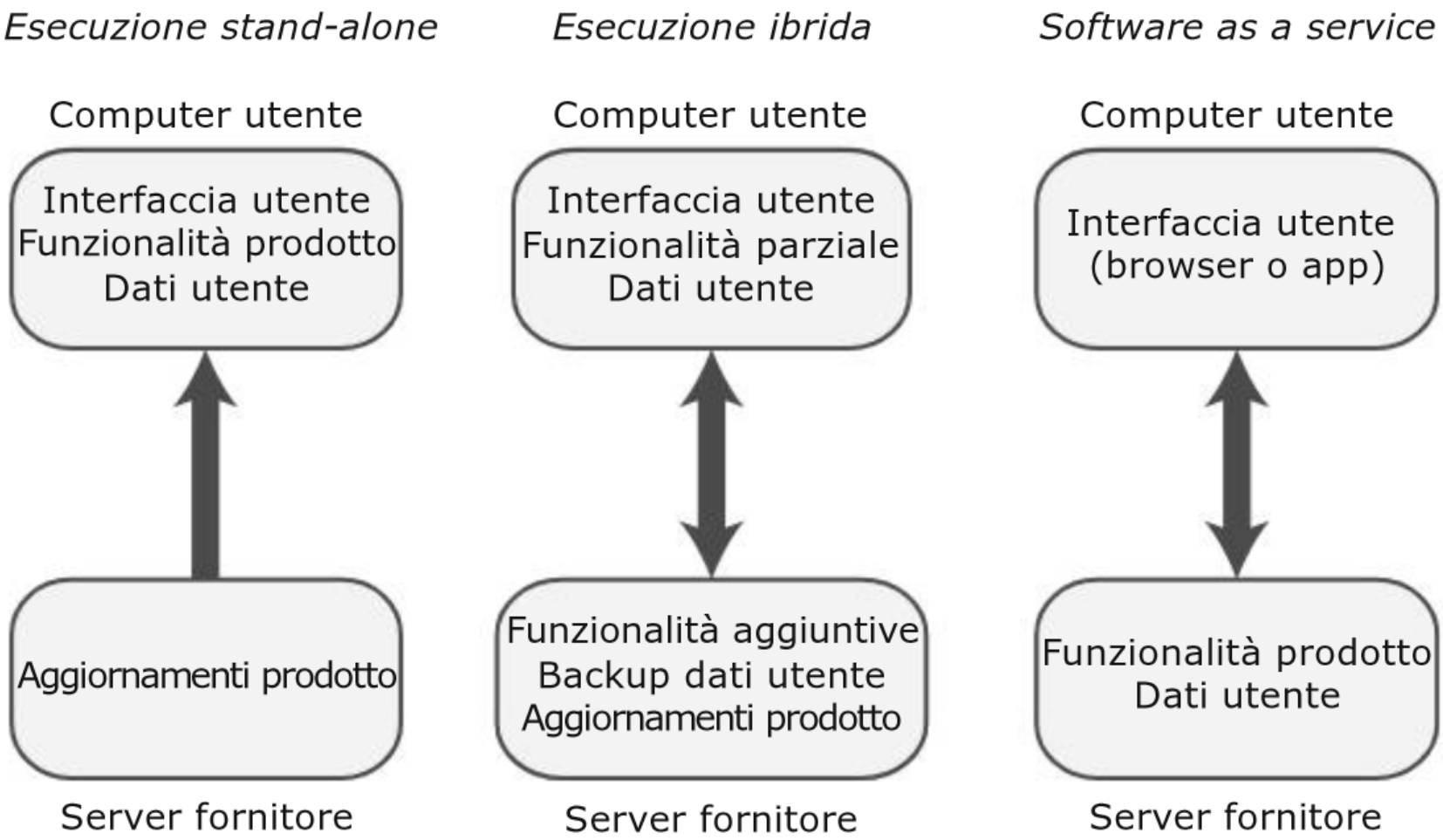
IS basata sul progetto



IS basata sul prodotto



Diversi modelli di *esecuzione*



Diversi *tipi* di sistemi software

- **… quindi ulteriori proprietà specifiche dei sistemi**
- **Sistemi informativi**
  - Per gestire informazioni
    - Sistemi bancari, sistemi bibliotecari, sistemi amministrativi
- **Sistemi in tempo reale**
  - Il sw risponde ad eventi esterni entro un periodo di tempo predefinito (e limitato)
    - Monitoraggio di impianti, sistemi di sorveglianza, di automazione
- **Sistemi distribuiti**
  - Macchine indipendenti collegate in rete (IoT)
- **Sistemi embedded**
  - Componenti sw che controllano componenti hardware
    - Usato in aerei, robot, elettrodomestici, automobili, cellulari, dispositivi medici, ecc
- Ma anche **sistemi adattativi, cloud, mobili, CPSs, ecc**

Le *caratteristiche* del prodotto SW

Differente dai tipi tradizionali di prodotti

- Intangibile:
  - difficile da descrivere e valutare
- Malleabile:
  - può essere trasformato e dotato di nuove funzionalità
- *Human intensive*:
  - non comporta nessun processo manifatturiero tradizionale

Definizioni

Processo di produzione

- La sequenza di attività svolte per progettare, realizzare, consegnare e modificare un prodotto SW

Ciclo di vita di un software

- L'insieme di stadi in cui il sistema viene a trovarsi

Modelli di processo

- Determinano l'organizzazione delle diverse fasi di sviluppo

Il processo software

Un insieme strutturato di **attività** necessarie per sviluppare un sistema software.

Molti processi software diversi ma tutti coinvolgono:

- **Specifica**: definizione di ciò che il sistema dovrebbe fare;
- **Progettazione e implementazione** – definizione dell'organizzazione del sistema e implementazione del sistema;
- **Convalida** – verifica che faccia ciò che il cliente vuole;
- **Evoluzione**: cambiare il sistema in risposta alle mutevoli esigenze dei clienti.

Un modello di processo software è una *rappresentazione astratta* di un processo

Descrizioni dei processi software

Le descrizioni dei processi possono anche includere:

- **Prodotti**, che sono i risultati di un'attività di processo;
- **Ruoli**, che riflettono le responsabilità delle persone coinvolte nel processo;
- **Pre** e **post-condizioni**, che sono affermazioni che sono vere prima e dopo che un'attività di processo è stata messa in atto o un prodotto prodotto.

Processi plan-driven e processi agili

- I processi **plan-driven** sono processi in cui tutte le attività di processo sono pianificate in anticipo e i progressi vengono misurati rispetto a questo piano.
- Nei **processi agili** (o sw-driven), la pianificazione è incrementale ed è più facile modificare il processo per riflettere le mutevoli esigenze dei clienti.
- In pratica, la maggior parte dei processi pratici include elementi di approcci sia plan-driven che agili.
- Non esistono processi software giusti o sbagliati ma dipende dal sw sviluppato e dall'azienda

Qualità del processo software

- Produttività:
  - misura l'efficienza del processo di produzione del SW in termini di *velocità di consegna* del SW
- Tempestività:
  - misura la capacità del processo di produzione del SW di *valutare e rispettare i tempi di consegna* del prodotto
- Trasparenza:
  - misura la capacità del processo di produzione del SW di *capire il suo stato attuale e tutti i suoi passi*

Modelli di processo

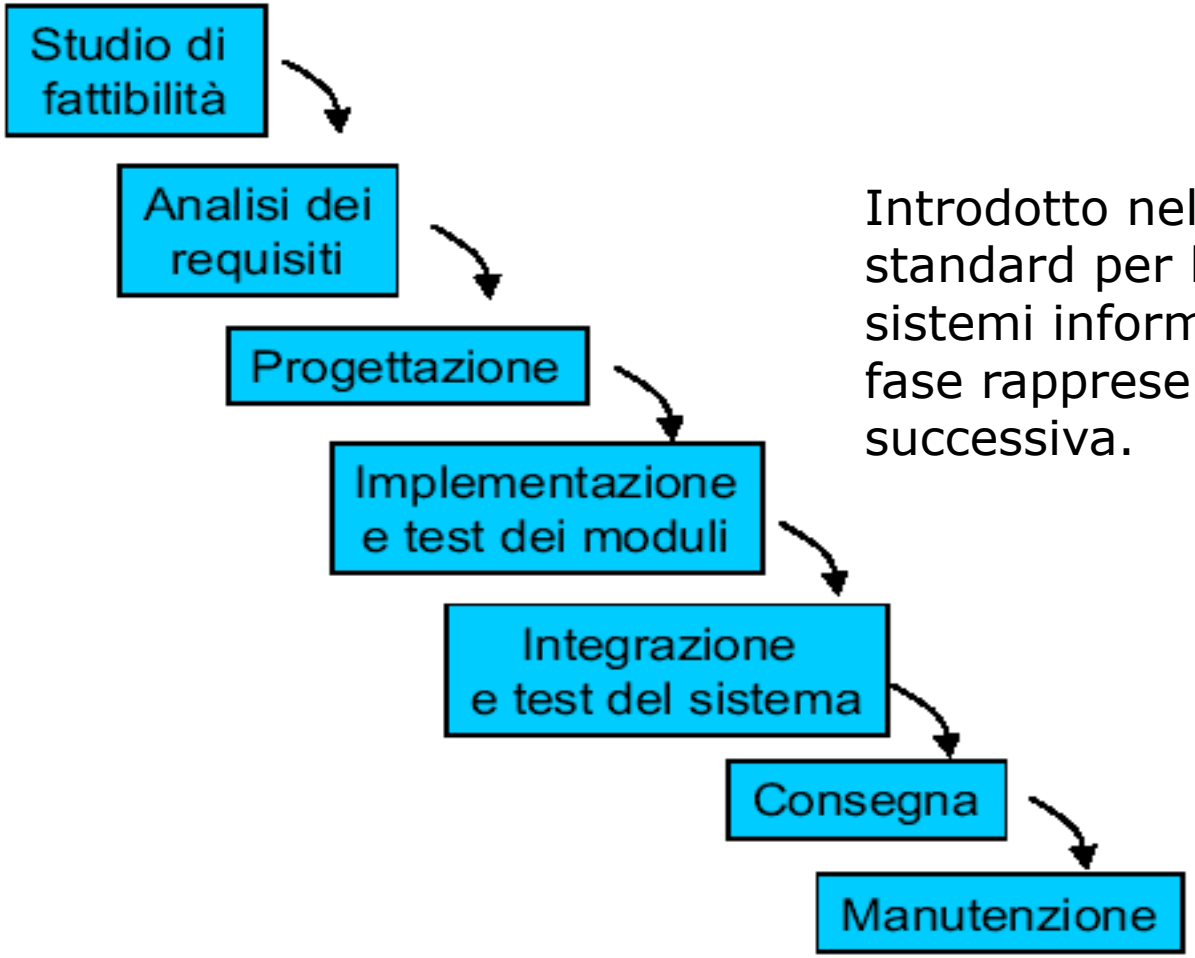
I modelli di ciclo di vita sono classificati in

- Modelli a cascata
- Modelli evolutivi o incrementali
- Modelli a spirale/V
- Modelli agili

Plan-driven

SW-driven

Modello a cascata (waterfall)



Introdotto nel 1970 rappresenta lo standard per la produzione di sistemi informatici. L'output di ogni fase rappresenta l'input della successiva.

Fasi di sviluppo

Le attività del processo sono organizzate in

- Studio di fattibilità
- Specifica
- Progettazione (o design)
- Implementazione e test dei moduli
- Integrazione e Test del sistema
- Consegna
- Manutenzione

Studio di fattibilità

Scopo di questa fase è la produzione di un documento detto *Feasibility Study Document* (FSD) che valuti i costi e i benefici della applicazione proposta

L' FSD deve contenere

- La definizione del problema
- Le possibili soluzioni e le relative motivazioni
- Per ognuna delle soluzioni proposte, la stima dei benefici, dei costi, delle risorse richieste e dei tempi di consegna

Specifica (o analisi dei requisiti) (1)

Ha lo scopo di determinare le <u>funzionalità</u> e le <u>proprietà</u> del SW in termini di performance, facilità d'uso, portabilità, facilità di manutenzione, ecc.

E' una fase cruciale del processo di sviluppo

La fase di specifica produce il RASD

- *Requirements Analysis and Specification Document*
  - A volte noto come *SW Requirement Specification (SRS)*

Specifica (o analisi dei requisiti) (5)

RASD

- Deve permette al cliente di verificare le caratteristiche specificate
- Deve consentire al progettista di procedere allo sviluppo dell'architettura del SW
- Proprietà richieste:
  - univocamente identificabile (Req ID) – spesso ID mista
  - facilmente comprensibile
  - preciso, completo, coerente, non ambiguo, tracciabile
  - modificabile
- Dovrebbe includere:
  - *user manual* preliminare
  - *system test plan* (*d*efinizione delle modalità di test del sistema)

Progettazione

Scopo di questa fase è la produzione di un documento contenente la descrizione dell' architettura del SW sia a livello *globale* che a livello dei *singoli* moduli

La fase di design produce il *Design Document*

- Definizione delle componenti (o moduli)
- Relazioni tra le componenti
- Interazione tra le componenti

Obbiettivi della progettazione

- Sviluppo concorrente e separazione delle responsabilità

Implementazione e test dei moduli

È la fase in cui i programmi vengono realizzati dai programmatori i quali si occupano di effettuare i test sulle funzionalità inerenti i singoli moduli

Per ogni componente si fornisce

- Codifica
- Documentazione
- Specifica dei test effettuati

Integrazione e test del sistema

Questa fase ha lo scopo di assemblare il codice prodotto dai diversi gruppi di programmatori e verificarne l'effettiva compatibilità risolvendo eventuali errori derivanti dall'interazione di due o più moduli

- Non sempre questa fase viene considerata concettualmente distinta dalla fase di implementazione
- Può richiedere l'interazione con l'HW (CPSs, sistemi embedded, medical devices,…)

Consegna

In questa fase il sistema viene distribuito agli utenti che ne verificano il funzionamento, individuando eventuali malfunzionamenti o dissimilarità rispetto alle specifiche di progetto

La consegna avviene in due fasi:

- ***Beta testing***: il SW viene distribuito ad un insieme selezionato di utenti allo scopo di effettuare test in casi reali. Gli errori riscontrati vengono corretti prima della distribuzione effettiva
- ***Distribution***: il SW viene definitivamente rilasciato agli utenti. Gli errori che vengono riscontrati vengono solitamente corretti nelle versioni successive o tramite l'utilizzo di appositi software correttivi (*patch*)

Manutenzione

È la fase che include tutti i cambiamenti (correzione + evoluzione) che seguono la distribuzione del SW

Spesso comporta più del 50% del costo totale del SW

- Stima dei contributi della EU alle compagnie SW
  - 80% del budget impegnato per progetti IT speso per il mantenimento

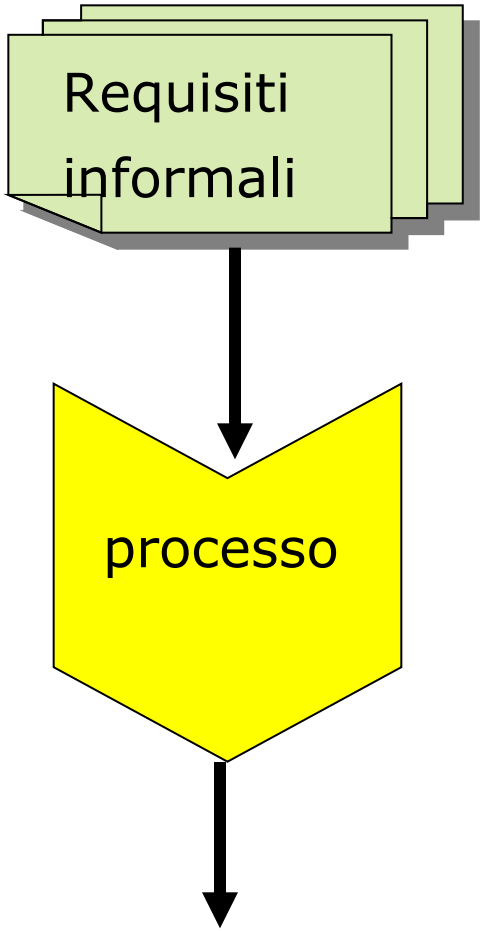
Classificazione:

- manutenzione **correttiva** ≈ 20%
- manutenzione **adattativa** ≈ 20%
- manutenzione **perfettiva** ≈ 50%

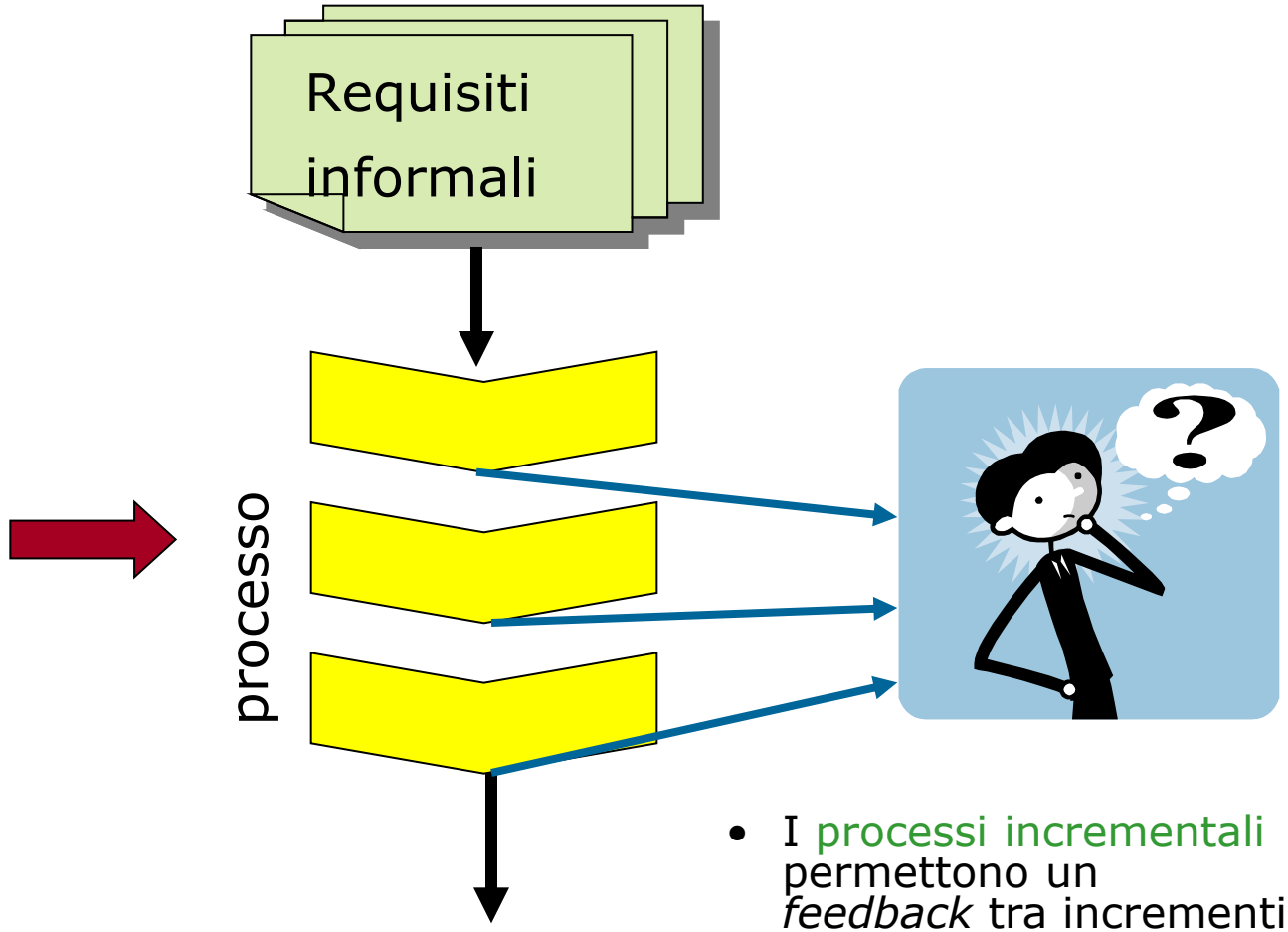
Modello a cascata: caratteristiche e limiti

- Ne esistono molte varianti e ciascuna organizzazione tende a definire il "proprio"
- Il modello implica il completamento di una fase prima di passare alla fase successiva
- È un processo black box e non consente prototipazione
- Non tiene in considerazione il fatto che i requisiti cambiano
  - Cambia il contesto
    - Hardware, dominio di applicazione, algoritmi
  - Specifiche sbagliate
    - Requisiti non catturati correttamente o incompleti
    - Scarsa conoscenza del dominio

Come affrontare l'evoluzione



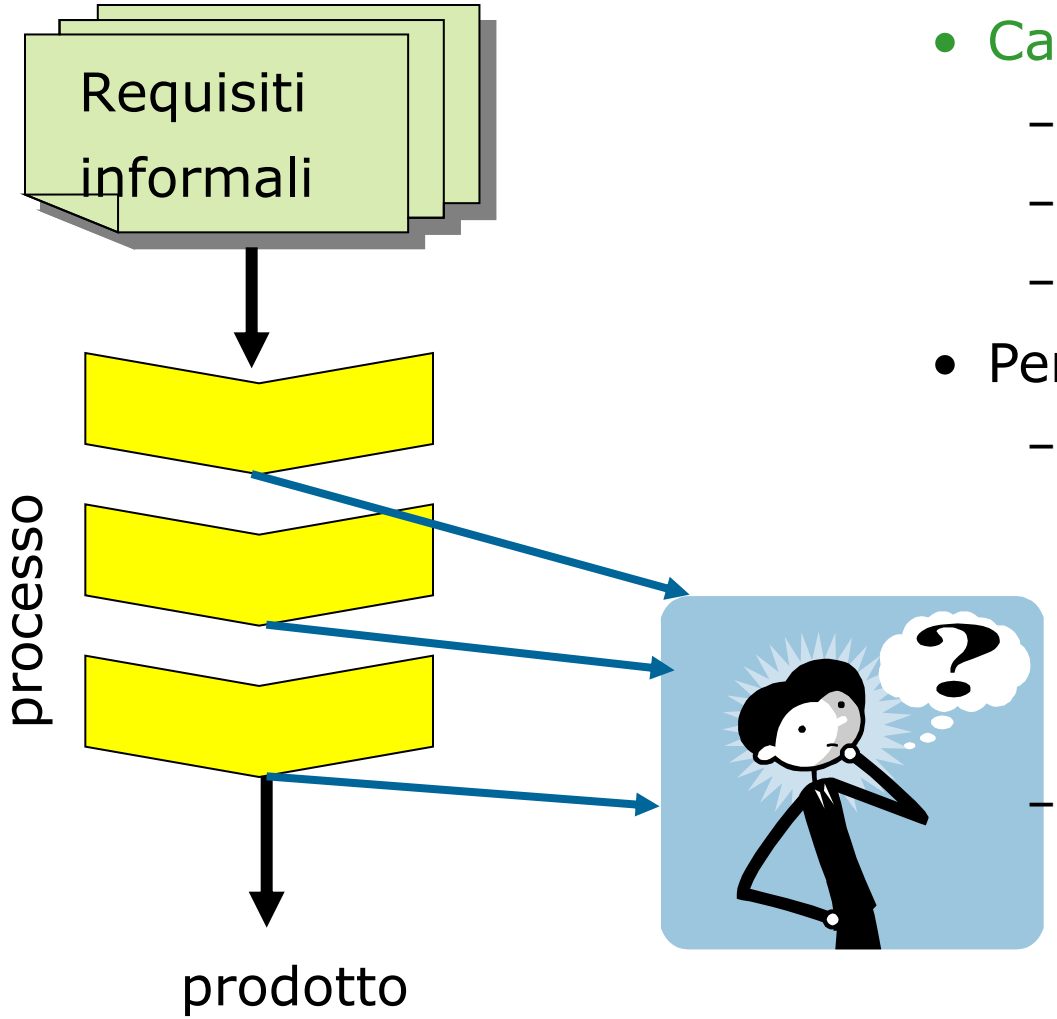
Modello black box



Modello incrementale

- I processi incrementali permettono un *feedback* tra incrementi successivi
- Il feedback è fondamentale per consentire *controlli* e gestire il *cambiamento*

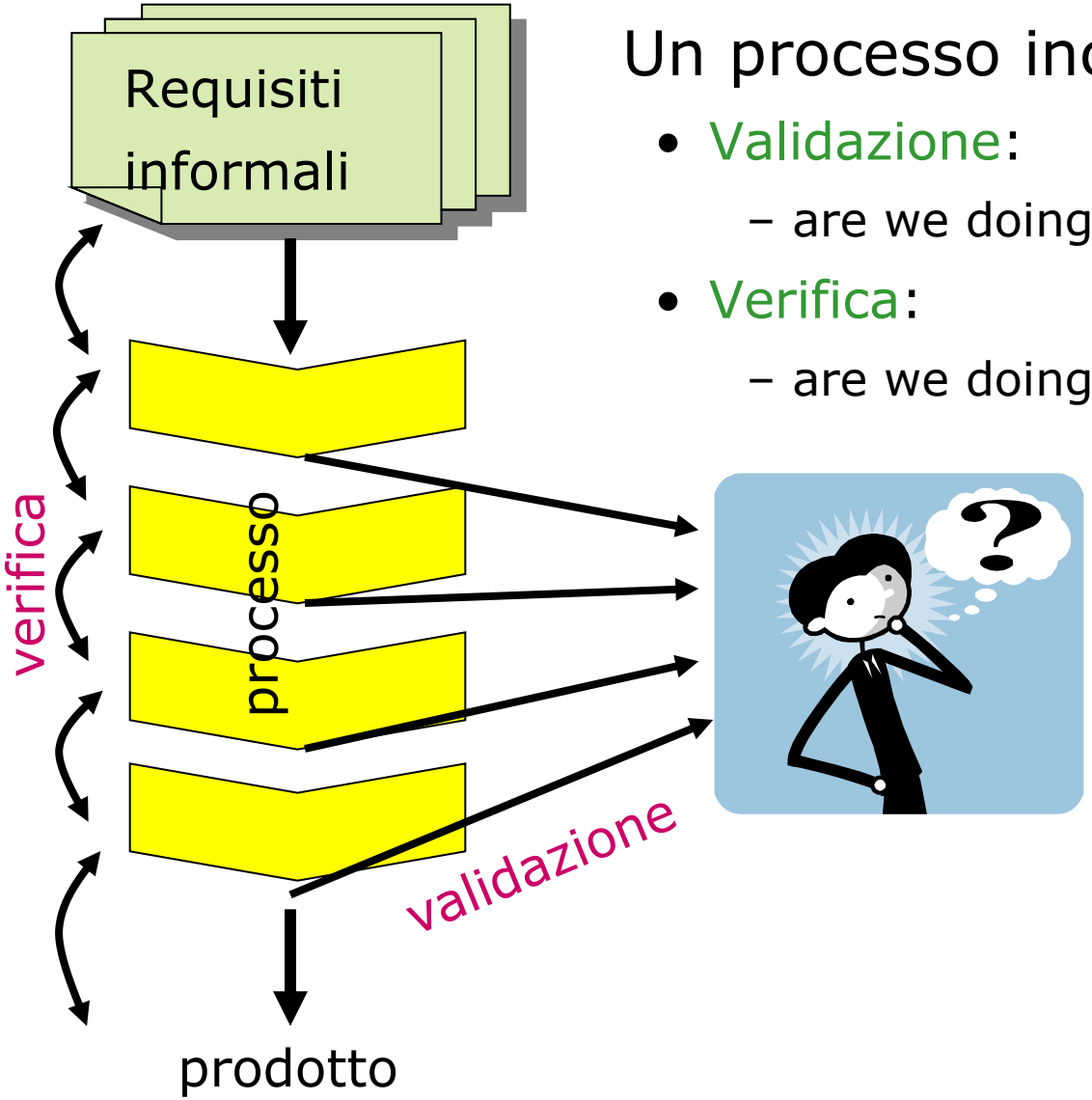
Processi incrementali



- Cambiamenti:
  - dei requisiti
  - della specifica
  - del design
- Permettono di:
  - Applicare i cambiamenti in maniera consistente in tutti i documenti
    - Spesso i cambiamenti sono applicati solo al codice
  - Modificare prima il progetto, e poi cambiare il codice

Modello incrementale

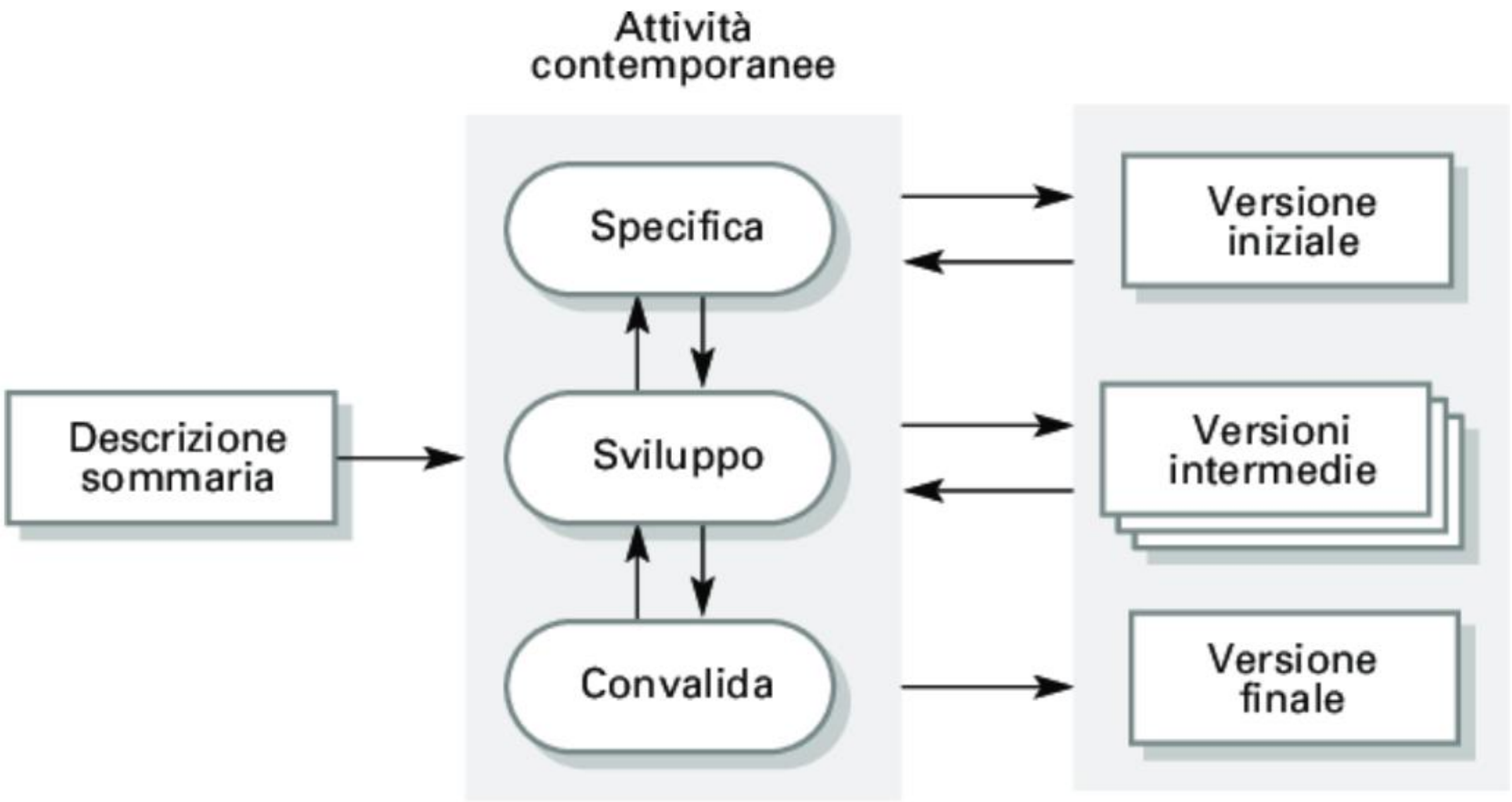
Validazione e Verifica



Un processo incrementale consente

- Validazione:
  - are we doing the right product?
- Verifica:
  - are we doing the product right?

Modello incrementale



Vantaggi dello sviluppo incrementale

Il costo per soddisfare le mutevoli esigenze dei clienti è ridotto.

- La quantità di analisi e documentazione che deve essere rifatta è molto inferiore a quella richiesta con il modello a cascata.

È più facile ottenere il feedback dei clienti sul lavoro di sviluppo che è stato fatto.

- I clienti possono commentare i prototipi del software e vedere quanto è stato implementato.

È possibile un'implementazione ed una consegna più rapida.

Limiti dello sviluppo incrementale

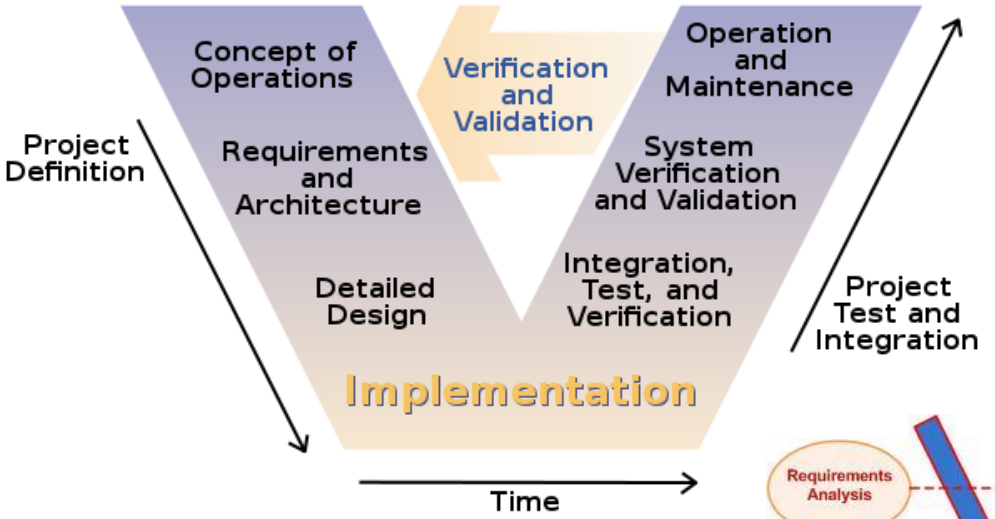
Il processo non è *trasparente*

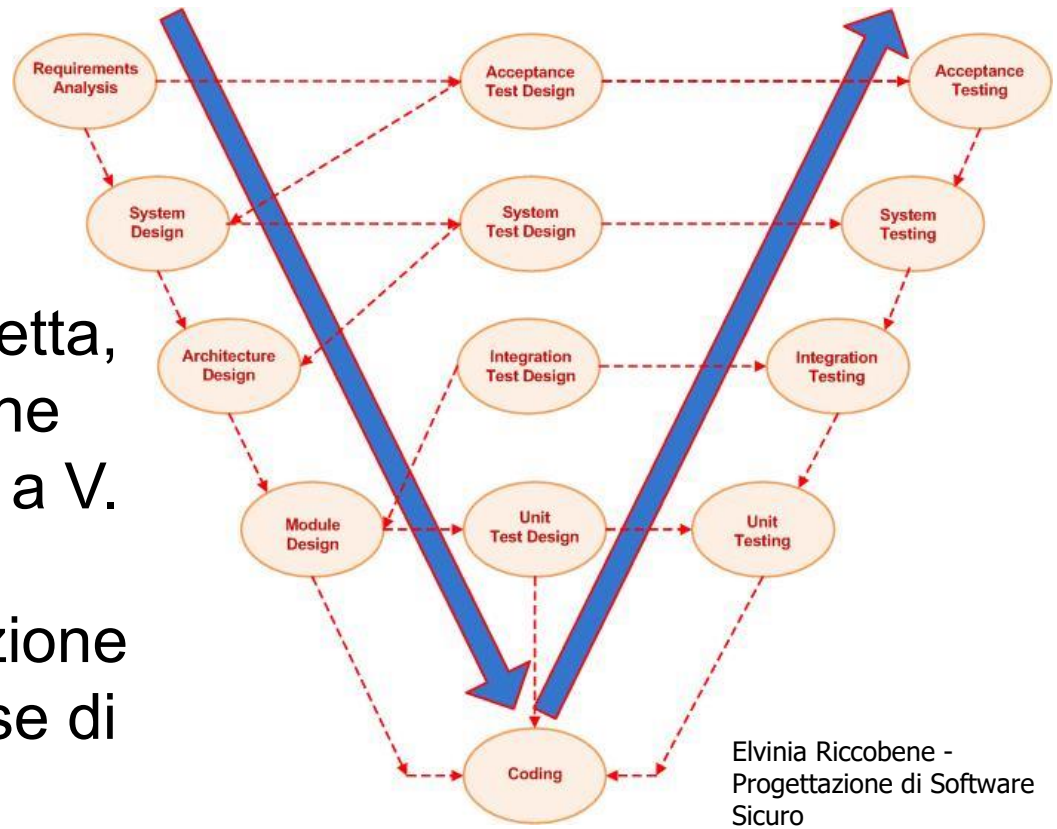
- Difficiltà a misurare i progressi
- Limitata documentazione sulle diverse versioni del sistema

La struttura del sistema tende a degradarsi man mano che vengono aggiunti nuovi incrementi

- Il cambiamento regolare tende a corrompere la sua struttura
  - Necessità di refactoring continuo
- Incorporare ulteriori modifiche al software diventa sempre più difficile e costoso

Modello a V



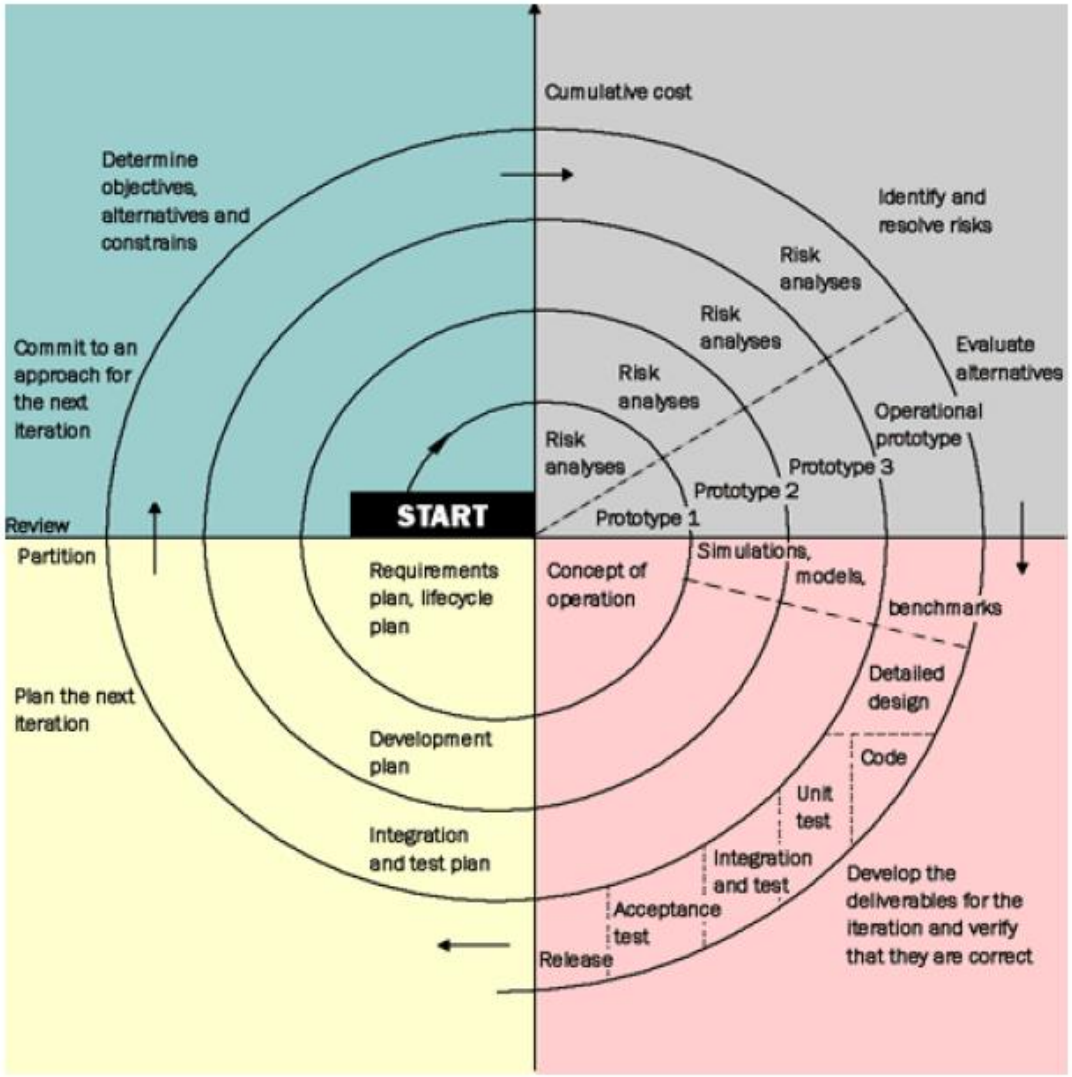


Questo modello estende il Modello a cascata: non discende lungo una linea retta, ma dopo la programmazione risale con una tipica forma a V.

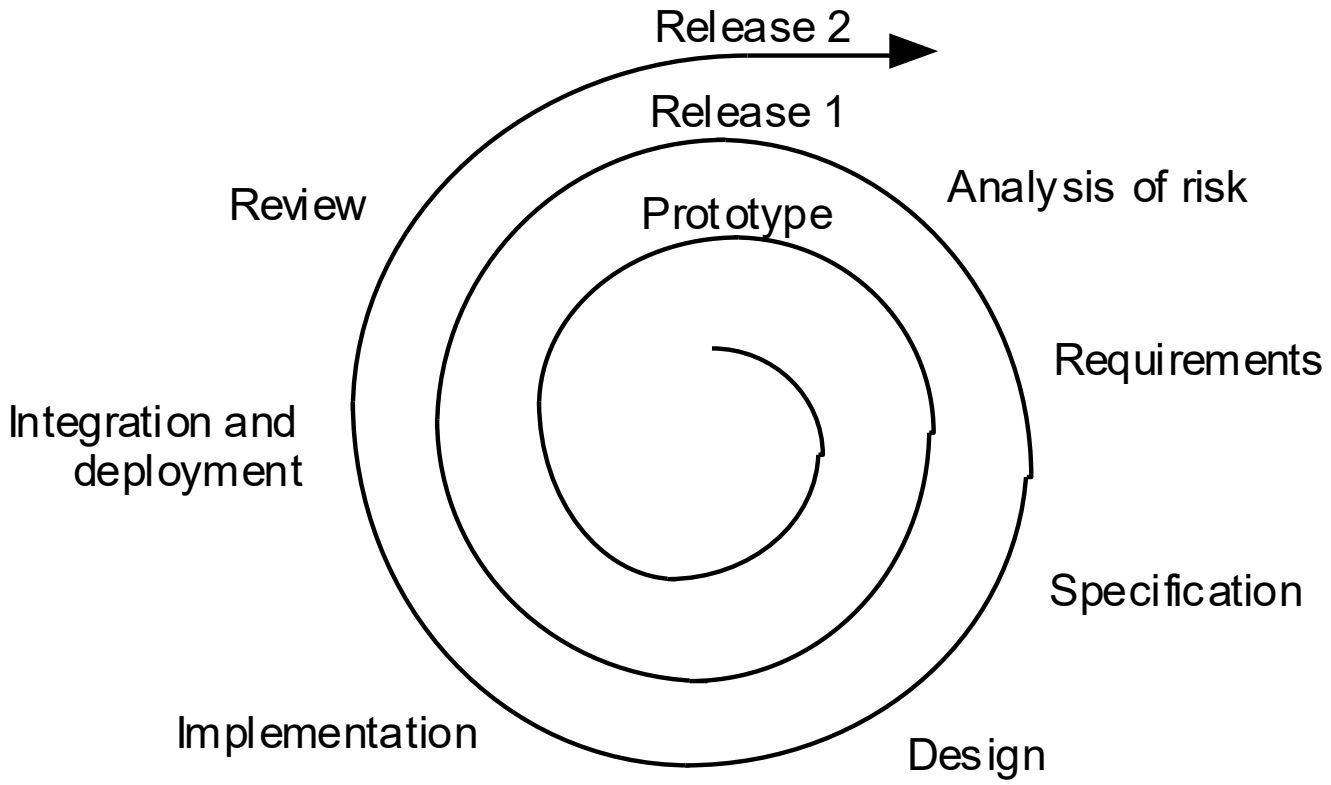
Il modello dimostra la relazione tra fase di ciclo di vita e fase di **testing**.

Modello a spirale (3)

- Ingloba *prototipazione* ed approccio *iterativo*
- Può essere visto come un *metamodello* di sviluppo
- Il mantenimento è una forma di sviluppo continuo



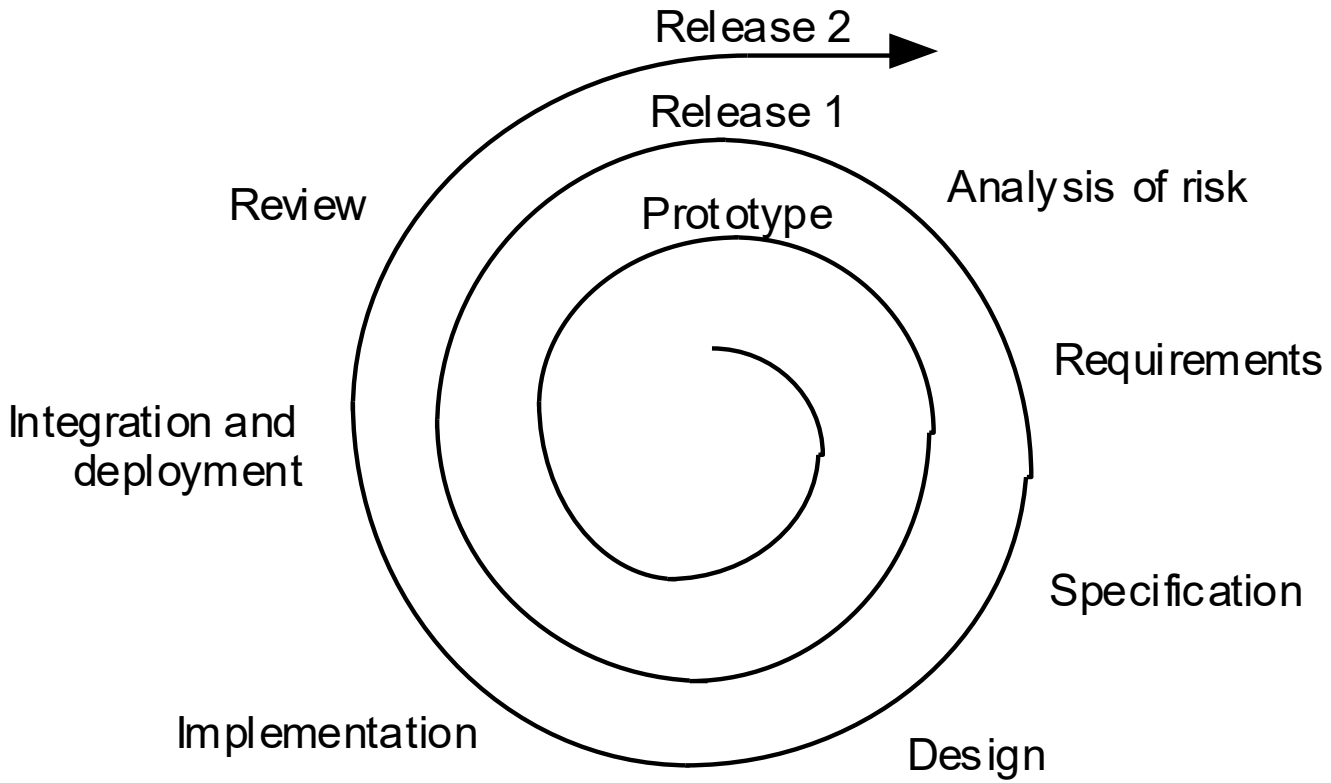
Modello a spirale (1)



E' un processo ciclico tra le seguenti attività:

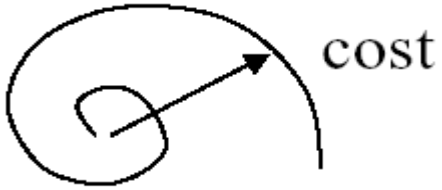
- analisi dei rischi, sviluppo (specifica, design, codifica), testing e revisione della release

Modello a spirale (2)

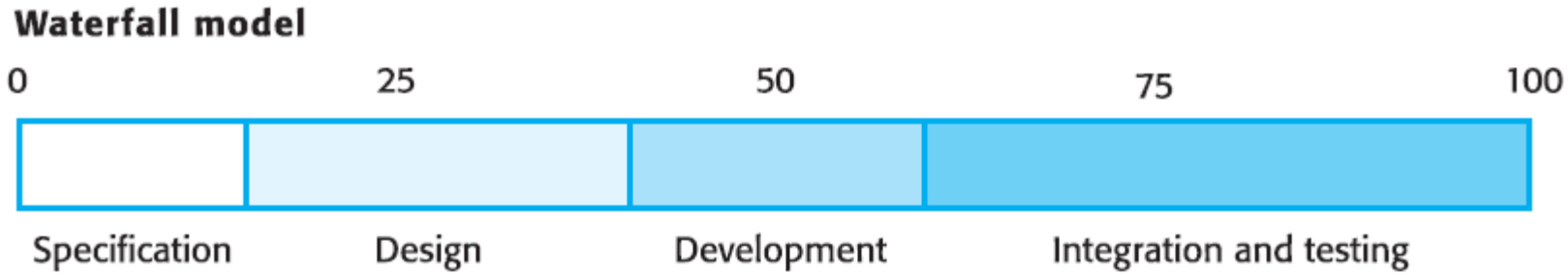


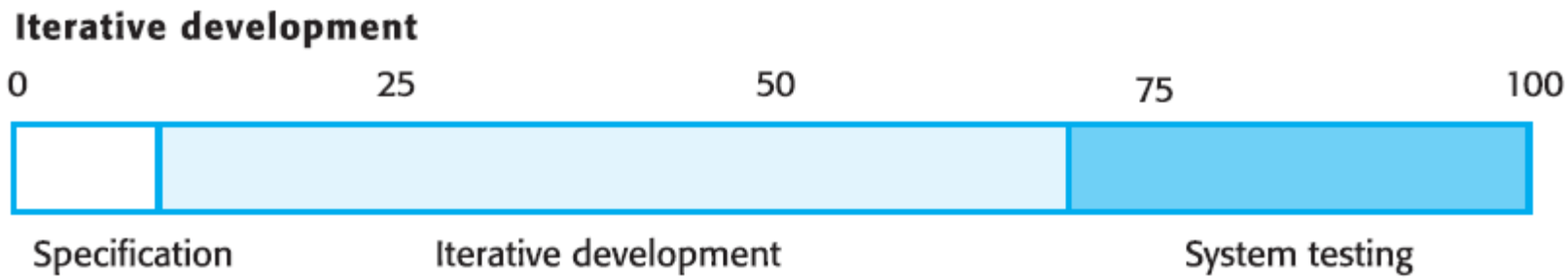
Ad ogni ciclo il costo aumenta

come in una spirale

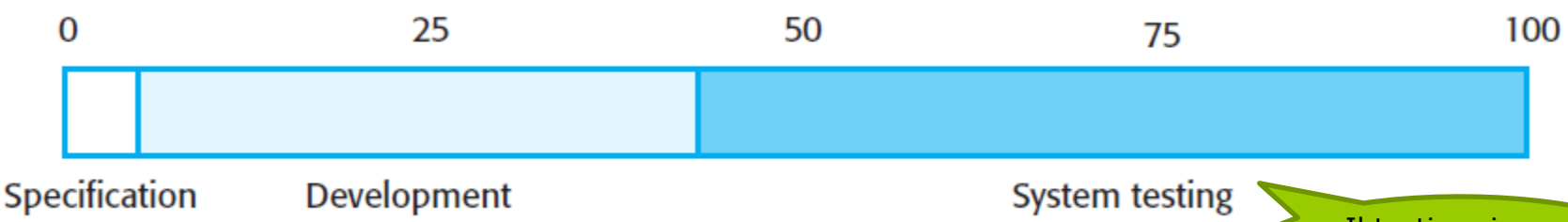


Confronto su costi di sviluppo





Mediamente….

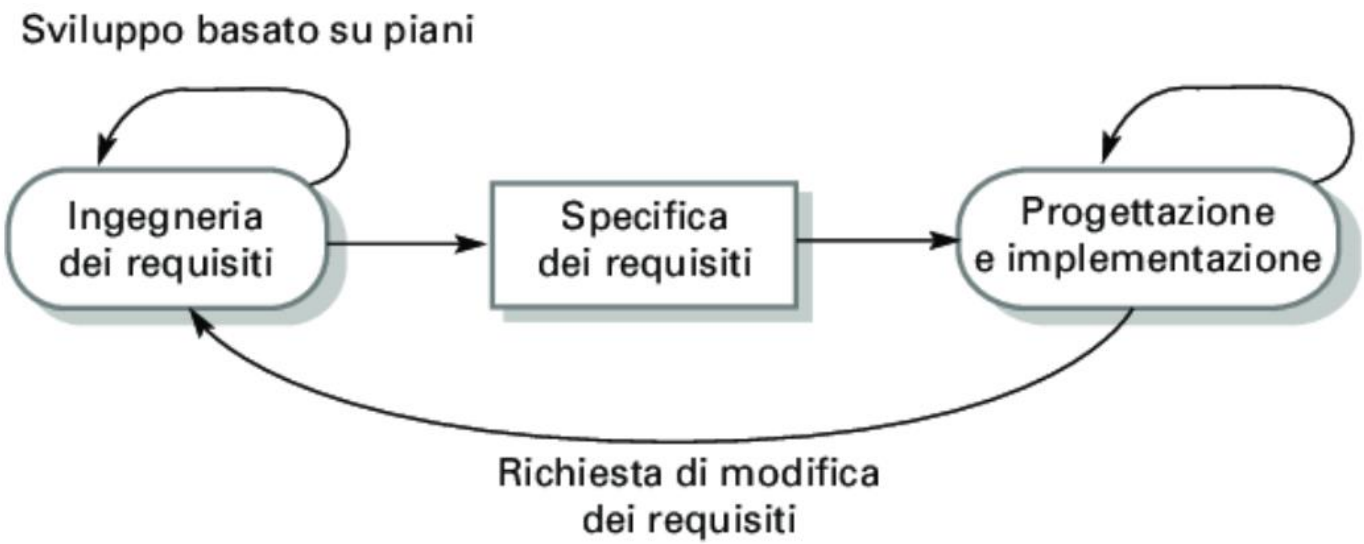


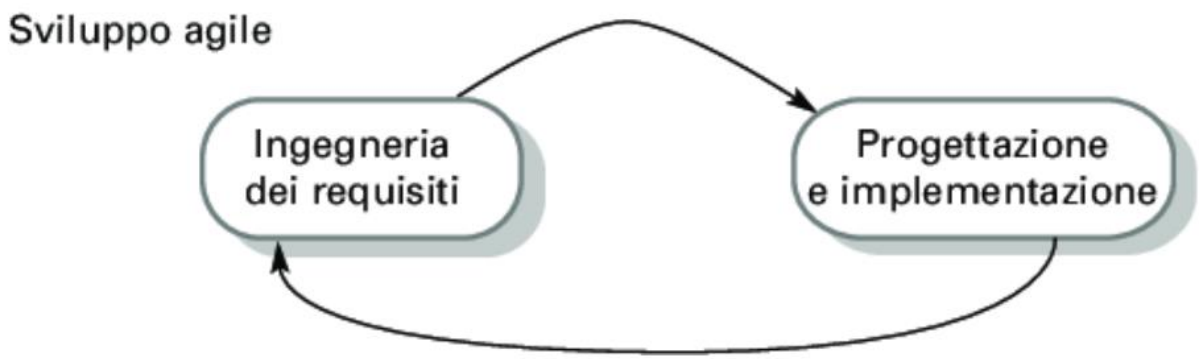
Il testing rimane l'attività molto costosa

Nuova tendenza

- **Metodologie pesanti** per i vecchi metodi come il *Modello a cascata*
- **Metodologie iterattive** per i metodi come il *Modello a spirale*
- **Metodologie agili** (o *leggere*) che coinvolgono quanto più possibile il committente, ottenendo in tal modo una elevata reattività alle sue richieste

Plan-driven vs SW-driven





Perchè *agile methods* e *continuous integration*

1. Spesso è praticamente impossibile derivare un set completo di requisiti software stabili (varie cause)
2. Processi di sviluppo software che prevedono di *specificare completamente* i requisiti e passare poi a *progettazione, implementazione e test* del sistema non sono idonei allo sviluppo rapido del software
   - Per i safety-critical control systems, in cui è essenziale un'analisi completa del sistema, un approccio pianificato è quello giusto
   - Per i processi di sviluppo aziendale che si concentrano sullo sviluppo e la consegna rapidi del software sono essenziali
3. La crisi sw degli anni 90 su un processo di sviluppo più *plan-driven* che *sw-driven* ha portato alla nascita di approcci **agili** ed al concetto di **continuous integration**
   - *Agile alliance*, una organizzazione no-profit creata allo scopo di diffondere le metodologie agili

Manifesto per lo Sviluppo Agile di Software

agilemanifesto.org

Il manifesto stabilisce:

- Gli **individui** e le **interazioni** più che i processi e gli strumenti
- Il **software funzionante** più che la documentazione esaustiva
- La **collaborazione col cliente** più che la negoziazione dei contratti
- **Rispondere al cambiamento** più che seguire un piano di sviluppo (no *design for change*)

Principi dei metodi agili

Principio	Descrizione
Coinvolgimento del cliente	I clienti devono essere strettamente coinvolti in tutto il processo di sviluppo. Il loro ruolo è fornire nuovi requisiti del sistema, dar loro priorità e valutare le iterazioni del sistema.
Accettare i cambiamenti	Prevedere i requisiti di sistema che potranno cambiare; quindi progettare il sistema in modo che possa accogliere tali cambiamenti.
Consegna incrementale	Il software viene sviluppato per incrementi; il cliente specifica i requisiti da includere in ogni incremento.
Mantenere la semplicità	Concentrarsi sulla semplicità sia nel software che si sviluppa sia nel processo di sviluppo. Se possibile, lavorare attivamente per eliminare le complessità del sistema.
Persone, non processi	Le capacità del team di sviluppo devono essere riconosciute e sfruttate. I membri del team devono essere lasciati liberi di sviluppare il software secondo i loro metodi di lavoro, senza processi prescrittivi.

Metodi agili

I **metodi agili** sono una famiglia di metodi di sviluppo che hanno in comune:

– **Rilasci frequenti** del prodotto sviluppato

– **Collaborazione** continua del team di progetto col **cliente**

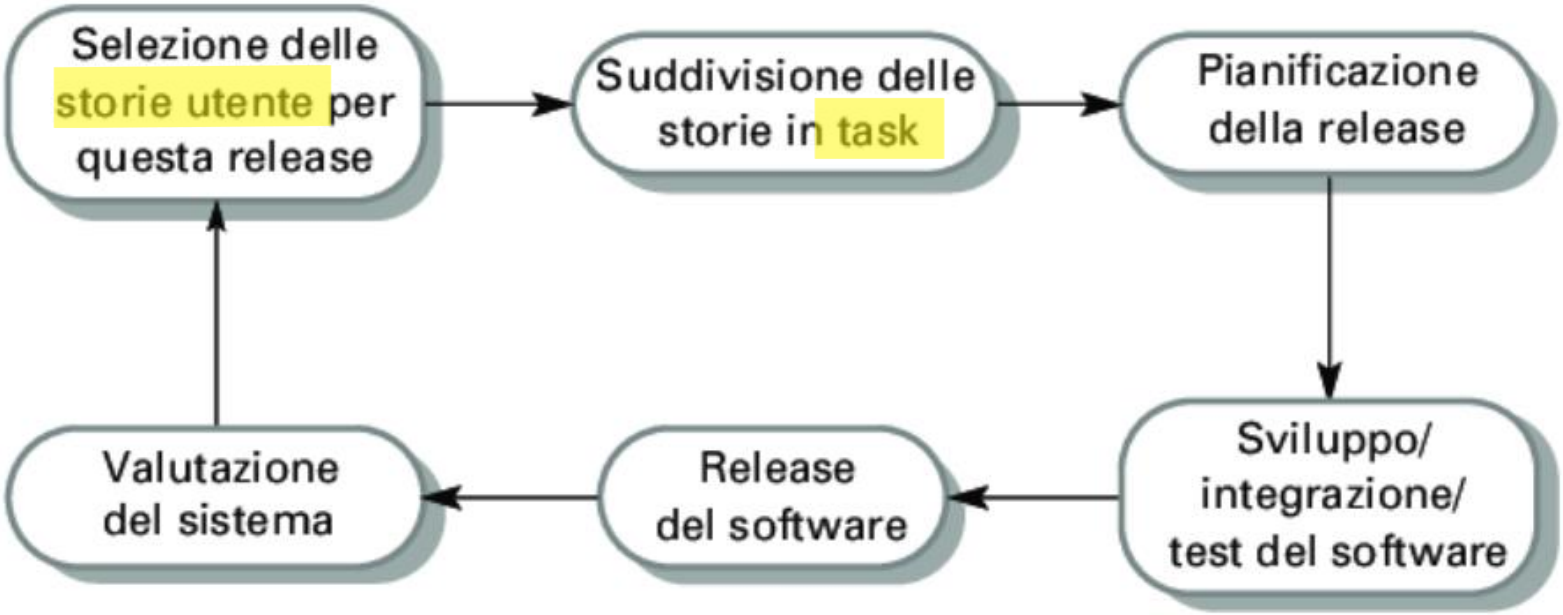
– **Documentazione** di sviluppo **ridotta**

– **Valutazione** sistematica e continua **di** valori e rischi **dei cambiamenti**

Esistono un certo numero di tali metodologie: Extreme Programming, SCRUM , ecc.

- Model-Driven Development come compromesso

Processo XP (programmazione estrema



Aspetti chiave:

- User stories (o *scenari*) per la specifica dei reqs
- Test-first development
- Pair programming
- Refactoring
- Continuous integration

Esempio di scenario o user story

Prescrizione dei farmaci
Kate è un dottore che desidera prescrivere un farmaco a un paziente che frequenta una clinica. La cartella clinica del paziente è già visualizzata sul suo computer, quindi fa clic sul campo del farmaco e può selezionare “farmaco corrente”, “nuovo farmaco” o “formulario”.
Se sceglie “farmaco corrente”, il sistema le chiede di controllare la dose. Se Kate vuole cambiare la dose, digita la nuova dose e conferma la prescrizione.
Se sceglie “nuovo farmaco”, il sistema suppone che Kate conosca quale farmaco prescrivere. Kate digita le prime lettere del nome del farmaco. Il sistema visualizza una lista di possibili farmaci che iniziano con quelle lettere. Kate seleziona il farmaco richiesto e il sistema risponde chiedendole di verificare se il farmaco selezionato è corretto. Lei digita la dose e conferma la prescrizione.
Se sceglie “formulario”, il sistema visualizza una finestra di ricerca per il formulario approvato. Kate può cercare il farmaco richiesto e poi lo seleziona. Il sistema le chiede di verificare se il farmaco scelto è corretto. Lei digita la dose e conferma la prescrizione.
Il sistema controlla sempre che la dose sia entro i limiti consentiti; in caso contrario, chiede a Kate di cambiare la dose.
Dopo che Kate ha confermato la prescrizione, dovrà controllarla e poi fare clic su “OK” o su “Cambia”. Se fa clic su “OK”, la prescrizione viene memorizzata nel database di controllo. Se fa clic su “Cambia”, il sistema riavvia il processo di prescrizione dei farmaci.

User story suddivisa in Tasks

Task 1: cambia la dose del farmaco prescritto
Task 2: scelta dal formulario
Task 3: controllo della dose

User story suddivisa in Tasks

Task 1: cambia la dose del farmaco prescritto
Task 2: scelta dal formulario
Task 3: controllo della dose
Il controllo della dose è una precauzione per verificare che il dottore non abbia prescritto una dose pericolosamente piccola o grande.
Utilizzando il codice del formulario come nome del farmaco, ricerca nel formulario le dosi minima e massima consigliate.
Confronta la dose prescritta con la minima e la massima. Se la dose è fuori dal range ammesso, visualizza un messaggio di errore per segnalare che la dose prescritta è troppo piccola o troppo grande. Se è all'interno del range, abilita il pulsante "Conferma".

Esempio di test iniziale

Task 4: controllo della dose

Input:

1. Un numero in mg rappresenta una singola dose di farmaco.
2. Un numero rappresenta il numero di singole dosi giornaliere.

Test:

1. Controlla l'input nel quale la singola dose è corretta, ma la frequenza è troppo alta.
2. Controlla l'input nel quale la singola dose * la frequenza è troppo grande o troppo piccola.
3. Controlla l'input nel quale la singola dose * la frequenza è nel range consentito.

Output:

OK o messaggio di errore per segnalare che la dose è fuori dal range consentito.

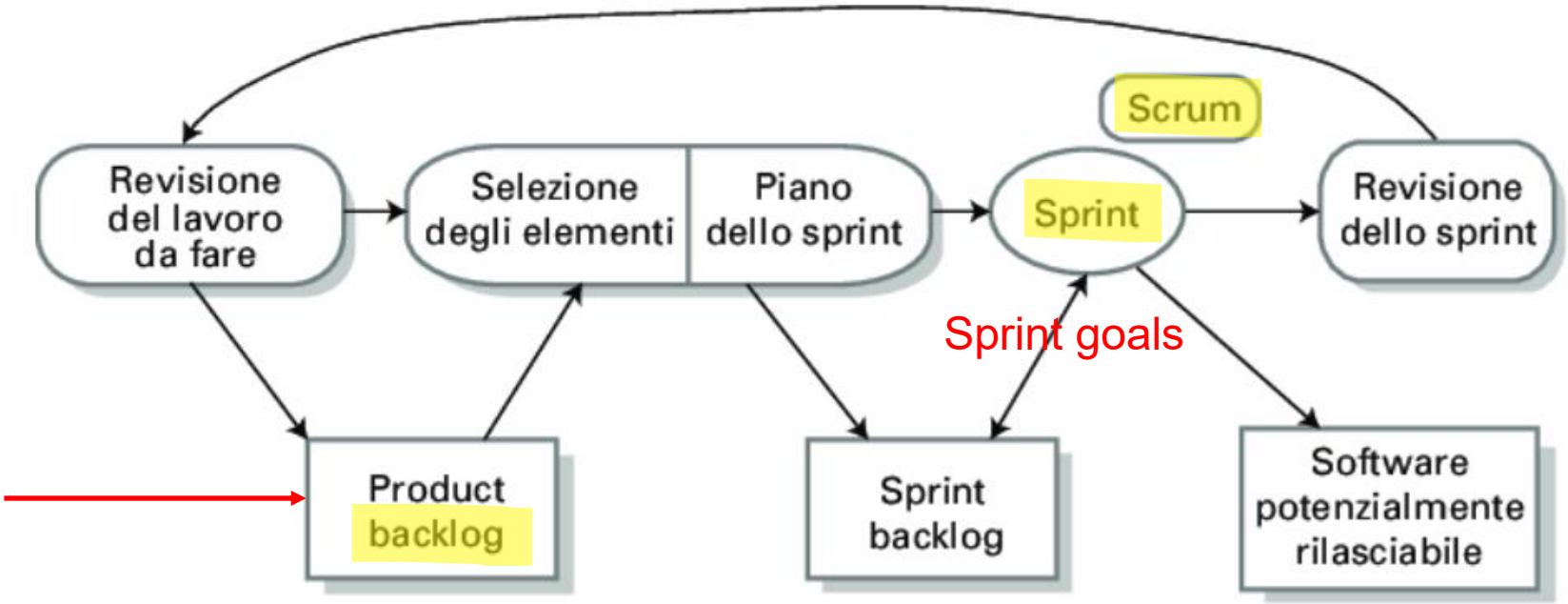
Esempi di refactoring

Il codice deve essere costantemente rifattorizzato.

- Riorganizzazione di una **gerarchia di classi** per rimuovere il codice duplicato.
- Riordinare e rinominare **attributi e metodi** per renderli più facili da capire.
- Sostituzione del sezioni di codice simili con **chiamate a metodi** definiti in una libreria di programma.

Gli ambienti di sviluppo di solito includono strumenti di refactoring.

Processo Scrum



- Backlog del prodotto: l'elenco di *elementi* (caratteristiche del prodotto, requisiti, miglioramenti tecnici) su cui il team Scrum deve lavorare
- Sprint è una iterazione dello sviluppo e dura circa 2-4 settimane (sprint cycle)
- Scrum (daily meeting) riunione per la revisione e pianificazione delle fasi successive

- Supporta il continuous integration
- L'intero team ha visibilità di tutto
- I customers vedono un on-time delivery degli avanzamenti/miglioramenti e forniscono feedback immediato

Limiti dei metodi agili

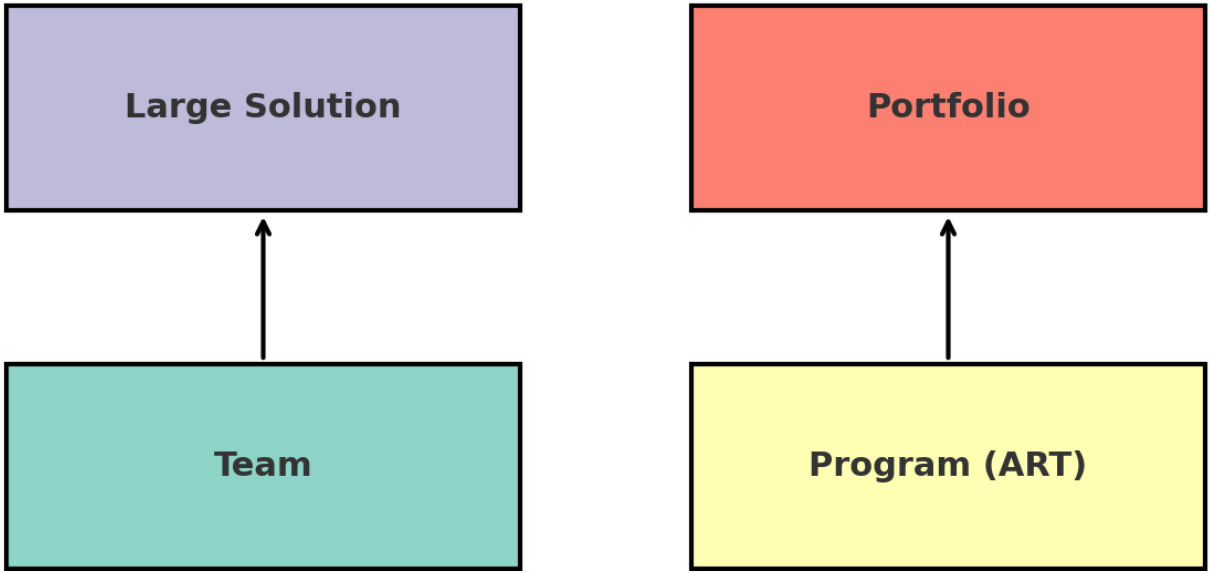
Non adeguato per software in sistemi embedded, e sistemi di grandi dimensioni/ complessi /critici

Numero di problemi:

1. L'informalità **vs** legalità aziendale/certificazione
2. nuovo sviluppo **vs** manutenzione
   - Mancanza di documentazione
   - Mantenere il cliente coinvolto
   - Continuità del team di sviluppo
3. piccoli team co-localizzati **vs** team distribuiti in tutto il mondo

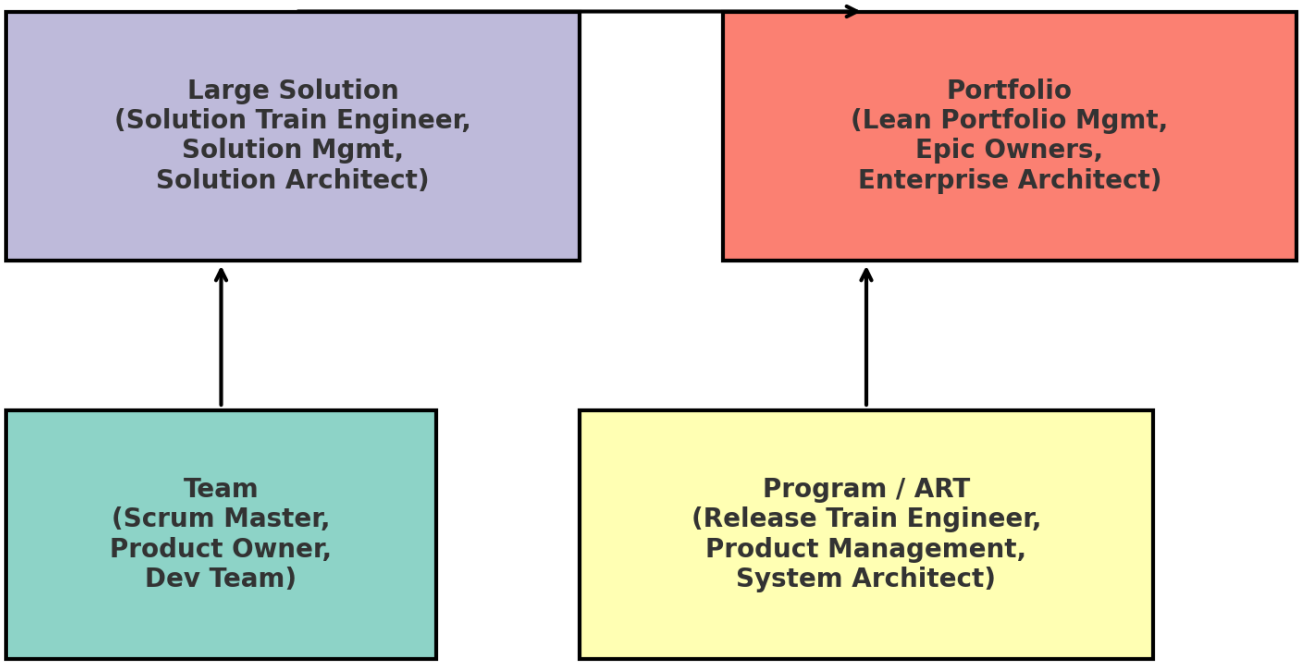
Fondamentale per la scalabilità del metodi agili è la loro integrazione in approcci plan-driven

Scaled Agile Framework (SAFe)



È un modello organizzativo e metodologico che serve ad **applicare i principi Agile su larga scala**, cioè non solo a livello di singolo team (come avviene con Scrum), ma anche a livello di **sviluppo di sistemi complessi e intera impresa** (di Dean Leffingwell)

Struttura dei livelli SAFe



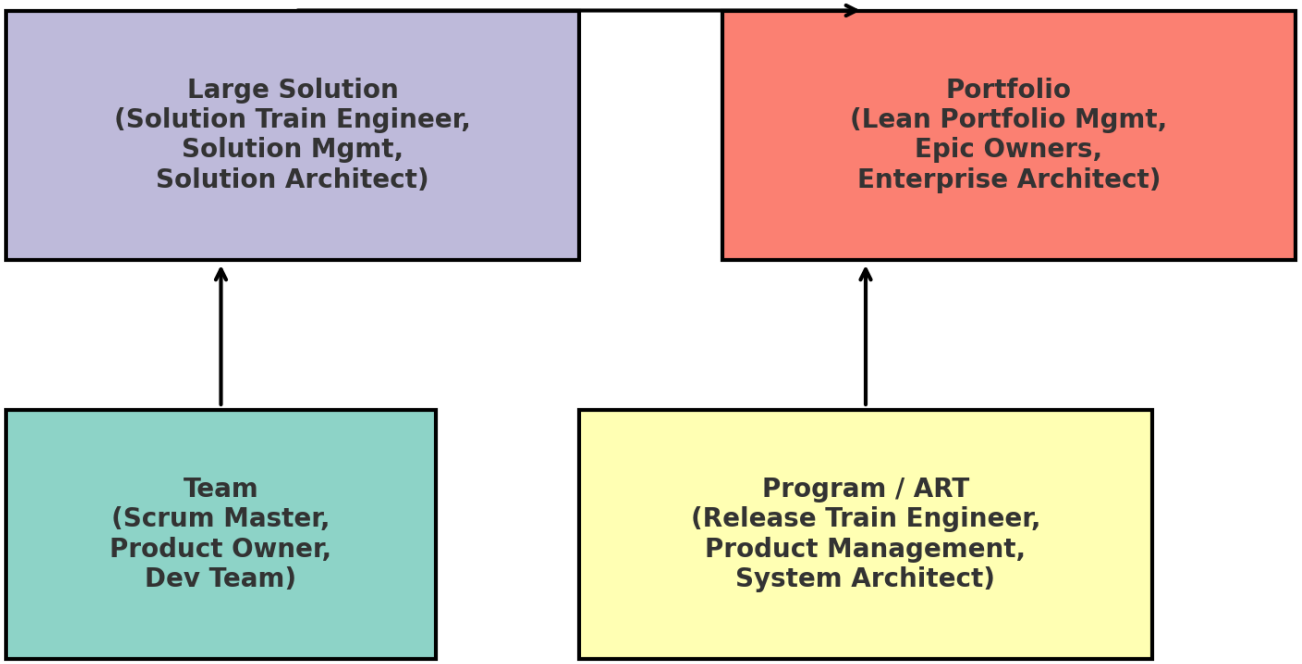
Team level

Team *Agile* classici, che lavorano con **Scrum o XP**

Producono incrementi del SW potenzialmente rilasciabili

Figura Chiave: Scrum Master

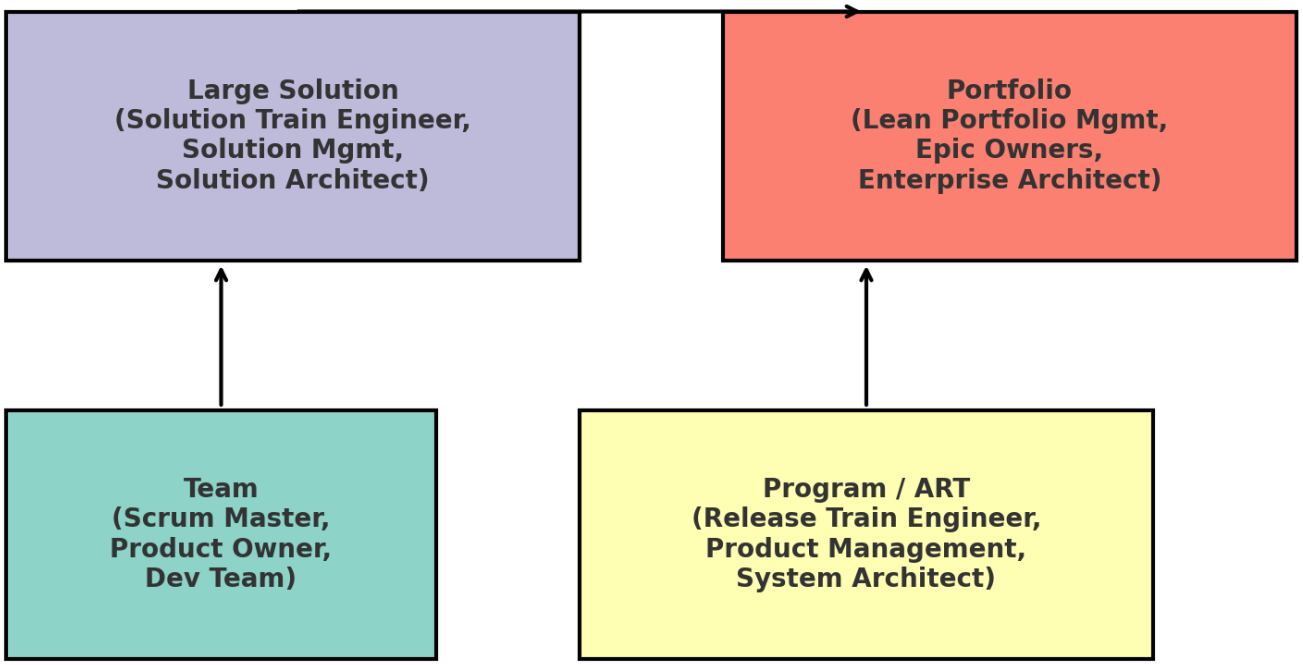
Struttura dei livelli SAFe



Program level

- Organizzazione di più team in un **Agile Release Train (ART)**: un *Agile train* è di 50–125 persone che lavorano insieme a un obiettivo comune
- Figura chiave: **Release Train Engineer** ("Scrum Master del train")
- Il lavoro viene pianificato in **Program Increments (PI)**, di 8–12 settimane

Struttura dei livelli SAFe

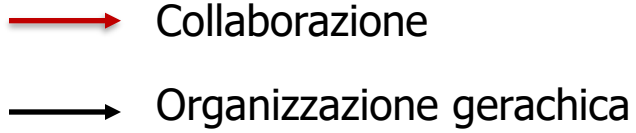


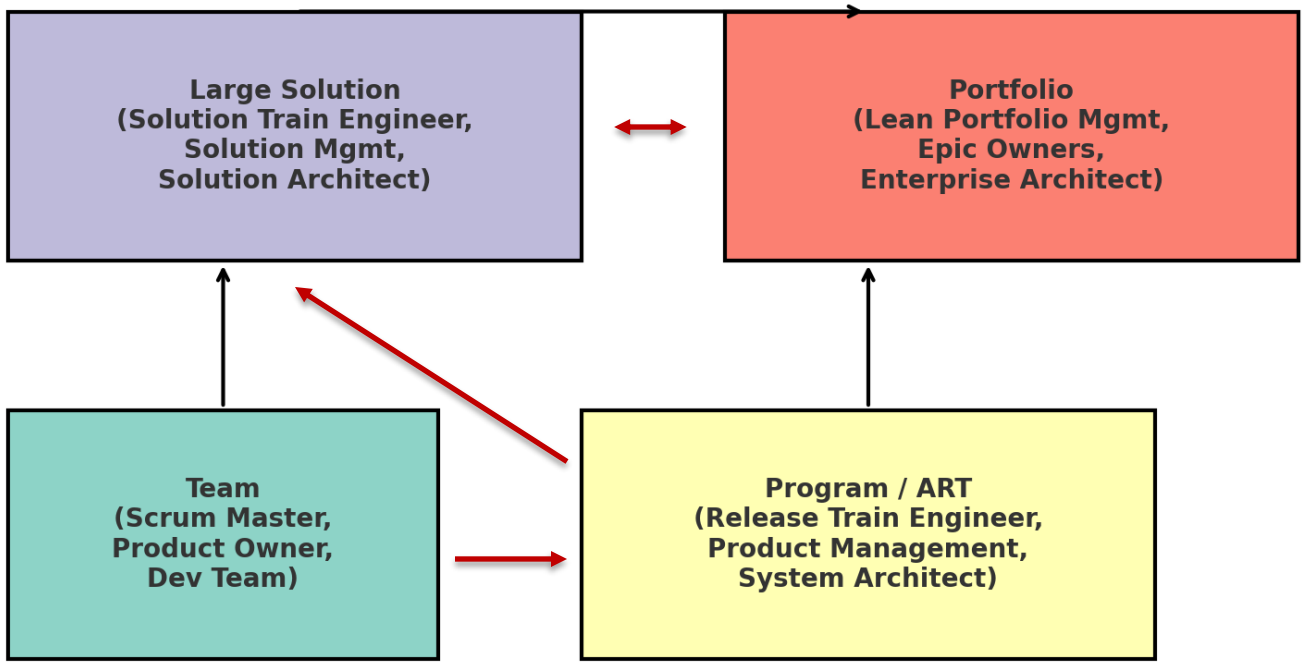
Large Solution level (opzionale, usato in contesti molto complessi)

Serve per coordinare più ART che insieme costruiscono grandi sistemi (per esempio nell'aerospazio, automotive, sanità)

Figura chiave: **Solution Train Engineer**

Struttura dei livelli SAFe





Portfolio level

Qui si decidono gli investimenti strategici e si gestisce il *flusso di valore* tramite **Value Streams**

- **flusso del valore:** insieme di passaggi (attività, persone, sistemi) necessari per passare dall'idea iniziale fino al rilascio e utilizzo del prodotto/servizio

Figura chiave: **Lean Portfolio Management**